

Expectations, Outcomes, and Challenges of Modern Code Review

Alberto Bacchelli
REVEAL @ Faculty of Informatics
University of Lugano, Switzerland
alberto.bacchelli@usi.ch

Christian Bird
Microsoft Research
Redmond, Washington, USA
cbird@microsoft.com

Abstract—Code review is a common software engineering practice employed both in open source and industrial contexts. Review today is less formal and more “lightweight” than the code inspections performed and studied in the 70s and 80s. We empirically explore the motivations, challenges, and outcomes of tool-based code reviews. We observed, interviewed, and surveyed developers and managers and manually classified hundreds of review comments across diverse teams at Microsoft. Our study reveals that while finding defects remains the main motivation for review, reviews are less about defects than expected and instead provide additional benefits such as knowledge transfer, increased team awareness, and creation of alternative solutions to problems. Moreover, we find that code and change understanding is the key aspect of code reviewing and that developers employ a wide range of mechanisms to meet their understanding needs, most of which are not met by current tools. We provide recommendations for practitioners and researchers.

I. INTRODUCTION

Peer code review, a manual inspection of source code by developers other than the author, is recognized as a valuable tool for reducing software defects and improving the quality of software projects [2], [1]. In 1976, Fagan formalized a highly structured process for code reviewing [13], based on line-by-line group reviews, done in extended meetings—*code inspections*. Over the years, researchers provided evidence on code inspection’s benefits, especially in terms of defect finding, but the cumbersome, time-consuming, and synchronous nature of this approach hinders its universal adoption in practice [32].

Nowadays many organizations are adopting more lightweight code review practices to limit the inefficiencies of inspections. In particular, there is a clear trend toward the usage of tools developed to support code review [28]. In the context of this paper, we define *Modern Code Review*, as review that is (1) informal (in contrast to Fagan-style), (2) tool-based, and that (3) occurs regularly in practice nowadays, for example at companies such as Microsoft, Google [19], Facebook [36], and in other organizations and open source software (OSS) projects [40].

This trend raises questions, such as: What are the expectations for code review nowadays? What are the actual outcomes of code review? What challenges do people face in code review?

Answers to these questions can provide insight for both practitioners and researchers. Developers and other software project stakeholders can use empirical evidence about expectations and outcomes to make informed decisions about when to use code review and how it should fit into their development process.

Researchers can focus their attention on the challenges faced by practitioners to make code review more effective.

We present an in-depth study of practices in teams that use modern code review, revealing what practitioners think, do, and achieve when it comes to modern code review.

Since Microsoft is made up of many different teams working on very diverse products, it gives the opportunity to study teams performing code review *in situ* and understand their expectations, the benefits they derive from code review, the needs they have, and the problems they face.

We set up our study as an exploratory investigation. We started without *a priori* hypotheses regarding how and why code review should be performed, with the aim of discovering what developers and managers expect from code review, how reviews are conducted in practice, and what the actual outcomes and challenges are. To that end, we (1) observed 17 industrial developers performing code review with various degrees of experience and seniority across 16 separate product teams with distinct reviewing cultures and policies; (2) interviewed these developers using a semi-structured interviews; (3) manually inspected and classified the content of 570 comments in discussions contained within code reviews; and (4) surveyed 165 managers and 873 programmers.

Our results show that, although the top motivation driving code reviews is finding defects, the practice and the actual outcomes are less about finding errors than expected: Defect related comments comprise a small proportion and mainly cover small logical low-level issues. On the other hand, code review additionally provides a wide spectrum of benefits to software teams, such as knowledge transfer, team awareness, and improved solutions to problems. Moreover, we found that context and change understanding is the key of any review. According to the outcomes they want to achieve, developers employ many mechanisms to fulfill their understanding needs, most of which are not currently met by any code review tool.

This paper makes the following contributions:

- Characterizing the motivations of developers and managers for code review and compare with actual outcomes.
- Relating the outcomes to understanding needs and discuss how developers achieve such needs.

Based on our findings, we provide recommendations for practitioners and implications for researchers as well as outline future avenues for research.

II. RELATED WORK

Previous studies have examined the practices of code inspection and code review. Stein *et al.* conducted a study focusing specifically on distributed, asynchronous code inspections [33]. The study included evaluation of a tool that allowed for identification and sharing of code faults or defects. Participants at separated locations can then discuss faults via the tool. Laitenburger conducted a survey of code inspection methods, and presented a taxonomy of code inspection techniques [22]. Johnson conducted an investigation into code review practices in OSS development and their effect on choices made by software project managers [18].

Porter *et al.* [26] reported on a review of studies on code inspection in 1995 that examined the effects of factors such as team size, type of review, and number of sessions on code inspections. They also assessed costs and benefits across a number of studies. These studies differ from ours in that they were not tool-based and were the majority involved planned meetings to discuss the code.

However, prior research also sheds light on why review today is more often tool-based, informal, and often asynchronous. The current state of code review might be due to the time required for more formal inspections. Votta found that 20% of the interval in a “traditional inspection” is wasted due to scheduling [38]. The ICICLE tool [11], or “Intelligent Code Inspection in a C Language Environment,” was developed after researchers at Bellcore observed how much time and work was expended before and during formal code inspections. Many of today’s review tools are based on ideas that originated in ICICLE. Other similar tools have been developed in an effort to reduce time for inspection and allow asynchronous work on reviews. Examples include CAIS [25] and Scrutiny [15].

More recently, Rigby has done extensive work examining code review practices in OSS development [29], [30], [28]. For example in a study of practices in the Apache project [29], they data-mined the email archives and found that reviews were typically small and frequent, and that the contributions to a review were often brief and independent from one another.

Sutherland and Venolia conducted a study at Microsoft regarding using code review data for later information needs [34]. They hypothesized that the knowledge exchanged during code reviews could be of great value to engineers later trying to understand or modify the discussed code. They found that “*the meat of the code review dialog, no matter what medium, is the articulation of design rationale*” and, thus, “*code reviews are an enticing opportunity for capturing design rationale.*”

When studying developer work habits, Latoza *et al.* found that many problems encountered by developers were related to understanding the rationale behind code changes and gathering knowledge from other members of their team [23].

III. METHODOLOGY

In this section we define the research questions, describe the research settings, and outline our research method.

A. Research Questions

Our investigation of code review revolves around the following research questions (iteratively refined during our initial in-field observations and interviews):

- 1) What are the motivations and expectations for modern code review? Do they change from managers to developers and testers?
- 2) What are the actual outcomes of modern code review? Do they match the expectations?
- 3) What are the main challenges experienced when performing modern code reviews relative to the expectations and outcomes?

B. Research Setting

Our study took place with professional developers, testers, and managers. Microsoft develops software in diverse domains, from high end server enterprise data management solutions such as SQL Server to mobile phone applications and smart phone apps to search engines. Each team has its own development culture and code review policies. Over the past two years, a common tool for code review at Microsoft has achieved wide-spread adoption. As it represents a common and growing solution for code review (over 40,000 developers used it so far), we focused on developers using this tool for code review—*CodeFlow*.

CodeFlow is a collaborative code review tool that allows users to directly annotate source code in its viewer and interact with review participants in a live chat model. The functionality of CodeFlow is similar to other review tools such as Google’s Mondrian [19], Facebook’s Phabricator [36] or open-source Gerrit [40]. Developers who want their code to be reviewed create a package with the changed (new, deleted, and modified) files, select the reviewers, write a message to describe the code review, and submit everything to the CodeFlow service. CodeFlow then notifies the reviewers about the incoming task via email.

Once reviewers open a CodeFlow review, they interact with it through a single desktop window (Figure 1). On the top left (1), they see the list of files changed in the current submission, plus a ‘description.txt’ file, which contains a textual explanation of the change, written by the author. On bottom left, CodeFlow shows the list of reviewers and their status (2). We see that Christian is the review author and Alberto, Tom, and Nachi are the reviewers. Alberto has reviewed and is waiting for the author to act, as the clock icon suggests, while Nachi already signed off on the changes. CodeFlow’s main view (3) shows the diff-highlighted content of the file currently under review. Both the reviewers and the author can highlight portions of the code and add comments inline (4). These comments can start threads of discussion and are the interaction points for the people involved in the review. Each user viewing the same review in CodeFlow sees events as they happen. Thus, if an author and reviewer are working on the review at the same time, the communication is synchronous and comment threads act similar to instant messaging. The comments are persisted so that if they work at different times, the communication

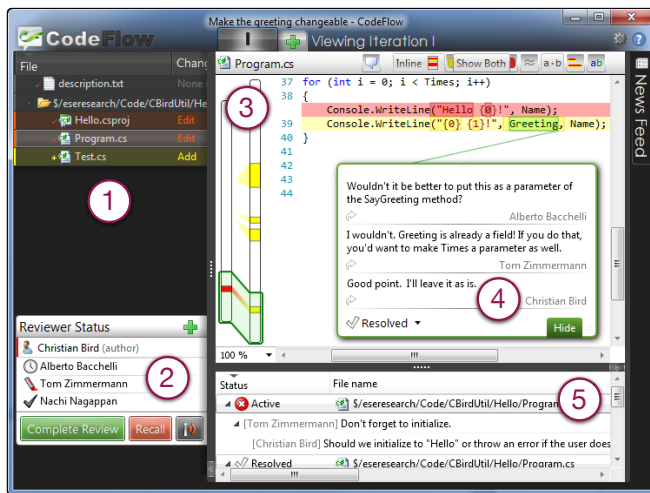


Fig. 1. CodeFlow, the main code review tool used by developers at Microsoft.

becomes asynchronous. The bottom right pane (5) shows the summary of all the comments in the review.

CodeFlow records all the information on code reviews on a central server. This provides an additional data source that we used to analyze real code review comments without incurring the Hawthorne effect [3].

C. Research Method

Our research method followed a mixed qualitative and quantitative approach [12] (depicted in Figure 2), which collects data from different sources for triangulation: (1) analysis of previous study, (2) observations and interviews with developers, (3) card sort on interview data, (4) card sort on code review comments, (5) the creation of an affinity diagram, and (6) survey to managers and programmers.

1. Analysis of previous study: Our research started with the analysis of a study commissioned by Microsoft, between April and May 2012 carried out by an external vendor. The study investigated how different product teams were using CodeFlow. It consisted of structured interviews (lasting 30-50 minutes) to 23 people with different roles.

Most of the interview questions revolved around topics that were very specific to usage of the tool, and were only tangentially related to this work. We found one relevant as a starting point for our study: “What do you hope to accomplish when you submit a code review?” We analyzed the transcript of this answer, for each interview, through the process of *coding* [9] (also used in *grounded theory* [4]): breaking up the answers into smaller coherent units (sentences or paragraphs) and adding *codes* to them. We organized codes into *concepts*, which in turn were grouped into more abstract *categories*.

From this analysis, four motivations emerged for code review: finding defects, maintaining team awareness, improving code quality, and assessing the high-level design. We used them to draw an initial guideline for our interviews.

2. Observations and interviews with developers: Subsequently, we conducted a series of one-to-one meetings with

developers who use CodeFlow, each taking 40-60 minutes.

We contacted 100 randomly selected candidates who signed-off between 50 and 250 code reviews since the CodeFlow release and sampled across different product teams to address our research questions from a *multi-point* perspective. We wrote developers who used CodeFlow in the past and asked them to contact us, giving us 30 minute notice when they received their next review task so that we could observe. The respondents that we interviewed comprised five developers, four senior developers, six testers, one senior tester, and one software architect. Their time in the company ranged from 18 months to almost 10 years, with a median of five years.

Each meeting was comprised of two parts: In the first part, we observed them performing the code review that they had been assigned. To minimize invasiveness and the Hawthorne effect, we used only one observer, and to encourage the participant to narrate their work, we asked the participants to consider him as a newcomer to the team. In this way, most developers thought aloud without need of prompting. With consent, we recorded the audio, assuring the participants of anonymity. Since we, as observers, have backgrounds in software development and practices at Microsoft, we could understand most of the work and where and how information was obtained without inquiry.

The second part of the meeting was a *semi-structured* interview [35]. This form of interviews makes use of an *interview guide* that contains general groupings of topics and questions rather than a pre-determined exact set and order of questions. They are often used in an exploratory context to “*find out what is happening* [and] *to seek new insights*” [39]. The guideline was iteratively refined after each interview, in particular when developers started providing answers very similar to the earlier ones, thus reaching a saturation effect.

After the first 5-6 meetings, the observations reached a *saturation* point [16]: They were providing insights very similar to the earlier ones. For this, we adjusted the meetings to have shorter observations, which we used as a starting point for our meetings and as a hook to talk about topics in our guideline.

The audio of each interview was then transcribed and broken up into smaller coherent units for subsequent analysis.

3. Card sort (meetings): To group codes that emerged from interviews and observations into categories, we conducted a *card sort*. Card sorting is a sorting technique that is widely used in information architecture to create mental models and derive taxonomies from input data [7]. In our case it helped to organize the codes into hierarchies to deduce a higher level of abstraction and identify common themes. A card sort involves three phases: In the (1) *preparation phase*, participants of the card sort are selected and the cards are created; in the (2) *execution phase*, cards are sorted into meaningful groups with a descriptive title; and in the (3) *analysis phase*, abstract hierarchies are formed to deduce general categories.

We applied an *open card sort*: There were no predefined groups. Instead, the groups emerged and evolved during the sorting process. In contrast, a closed card sort has predefined groups and is typically applied when themes are known in advance, which was not the case for our study.

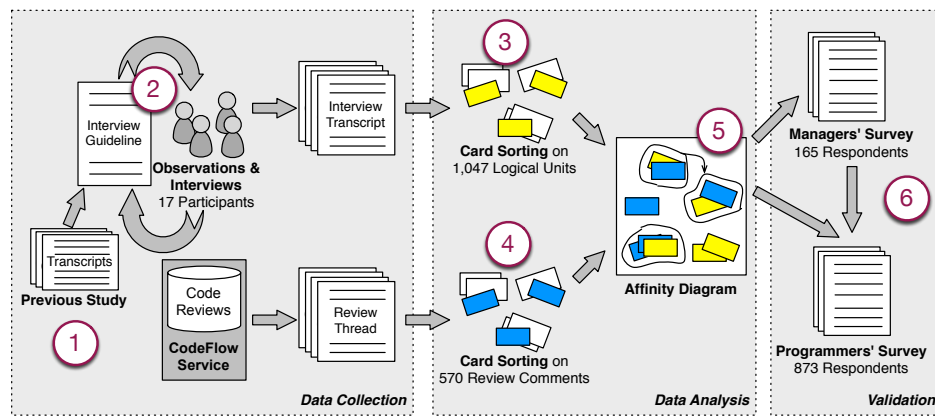


Fig. 2. The mixed approach research method applied.

The first author of this paper created all of the cards, from the 1,047 coherent units in the interviews (an example card is shown in a technical report [6]). Throughout our further analysis other researchers (the second author and external people) were involved in developing categories and assigning cards to categories, so as to strengthen the validity of the result. The first author played a special role of ensuring that the context of each question was appropriately considered in the categorization, and creating the initial categories. To ensure the integrity of our categories, the cards were sorted by the first author several times to identify initial themes. To reduce bias from the first author sorting the cards to form initial themes, all researchers reviewed and agreed on the final set of categories.

4. Card sort (code review comments): The same method was applied to group code review comments into categories: We randomly sampled 200 threads with at least two comments (e.g., Point 4 of Figure 2), from the entire dataset of CodeFlow reviews, which embeds data from dozens of independent software products at Microsoft. We printed one card for each comment (along with the entire discussion thread to give the context), totaling 570 cards, and conducted a card sort, as performed for the interviews, to identify common themes.

5. Affinity Diagram: We used an *affinity diagram* to organize the categories that emerged from the card sort. This tool allows large numbers of ideas to be sorted into groups for review and analysis [31]. We used it to generate an overview of the topics that emerged from the card sort, in order to connect the related concepts and derive the main themes. For generating the affinity diagram, we followed the five canonical steps: we (1) recorded the categories on post-it-notes, (2) spread them onto a wall, (3) sorted the categories based on discussions, until all are sorted and all participants agreed, (4) named each group, and (5) captured and discussed the themes.

6. Surveys: The final step of our study was aimed at validating the concepts that emerged from the previous phases. Towards this goal, we created two surveys to reach a significant number of participants and to challenge our conclusions (the full surveys are available as a technical report [6]). For the design of the surveys, we followed Kitchenham and Pfleeger's guidelines for personal opinion surveys [20]. Both surveys

were anonymous to increase response rates [37].

We sent the first survey to a cross section of managers. We considered managers for which at least half of their team performed code reviews regularly (on average, one per week or more) and sampled along two dimensions. The first dimension was whether or not the manager had participated in a code review himself since the beginning of the year and the second dimension was whether the manager managed a single team or multiple teams (a manager of managers). Thus, we had one sample of first level managers who participated in review, another sample of second level managers who participated in reviews, etc. The first survey was a short survey comprising 6 questions (all optional), which we sent to 600 managers that had at least 10 direct or indirect reporting developers who used CodeFlow. The central focus was the open question asking to enumerate the main motivations for doing code reviews in their team. We received 165 answers (28% response rate), which we analyzed before devising the second survey.

The second survey comprised 18 questions, mostly closed with multiple choice answers, and was sent to 2,000 randomly chosen developers who signed off on average at least one code review per week since the beginning of the year. We used the time frame of January to June of 2012 to minimize the amount of organizational churn during the time period and identify employees' activity in their current role and team. We received 873 answers (44% response rate). Both response rates were high, as other online surveys in software engineering have reported response rates ranging from 14% to 20% [27].

IV. WHY DO PROGRAMMERS DO CODE REVIEWS?

Our first research question seeks to understand what motivations and expectations drive code reviews, and whether managers and developers share the same opinions.

Based on the responses that we coded from observations of developers performing code review as well as interviews, there are various motivations for code review. Overall, the interviews revealed that finding defects, even though prominent, is just one of the many motivations driving developers to perform code reviews. Especially when reinforced by a strong team culture around reviews, developers see code reviews as an activity that has multiple beneficial influences not only on the code,

but also for the team and the entire development process. In this vein, one senior developer's comment summarized many of the responses: "[code review] *also has several beneficial influences: (1) makes people less protective about their code, (2) gives another person insight into the code, so there is (3) better sharing of information across the team, (4) helps support coding conventions on the team, and [...] (5) helps improving the overall process and quality of code.*"

Through the card sort on both meetings and code review comments, we found several references to motivations for code review and identified six main topics. To complete this list, in the survey for managers, we included an open question on why they perform code reviews in their team. We analyzed the responses to create a comprehensive list of high-level motivations. We included this list in the developers' survey and asked them to rank the top three main reasons that described why they do code reviews.

In the rest of this section, we discuss the motivations that emerged as the most prominent. We order them according to the importance they were given by the 873 developers and testers who responded to the final survey.

A. Finding Defects

One interviewed senior tester explains that he performs code reviews because they *"are a great source of bugs;"* he goes even further stating: *"sometimes code reviews are a cheaper form of bug finding than testing."* Moreover, the tool seems not to have an impact on this main motivation: *"using CodeFlow or using any other tool makes a little difference to us; it's more about being able to identify flaws in the logic."*

Almost all the managers included *finding defects* as one of the reasons for doing code reviews; for 44% of the managers, it is the top reason. Managers considered defects to be both low level issues (e.g., *"correct logic is in place"*) and high level concerns (e.g., *"catch errors in design"*). Concerning surveyed developers/testers, *finding defects* is the first motivation for code review for 383 of the programmers (44%), second motivation for 204 (23%), and third for 96 (11%).

This is in-line with the reason why code inspections were devised in the first place: reducing software defects [2].

Nevertheless, even though *finding defects* emerged from our data as a strong motivation (the first for almost half of the programmers and managers), interviews and survey results indicate that this only tells part of the story of why practitioners do code reviews and the outcomes they expect.

B. Code Improvement

Code improvements are comments or changes about code in terms of readability, commenting, consistency, dead code removal, etc., but do not involve correctness or defects.

Programmers ranked *code improvement* as an important motivation for code review, close to *finding defects*: This is the primary motivation for 337 (39%) programmers, the second for 208 (24%), and the third for 135 (15%). Managers reported *code improvement* as their primary motivation in 51 (31%) cases. One manager wrote how code review in her view is a

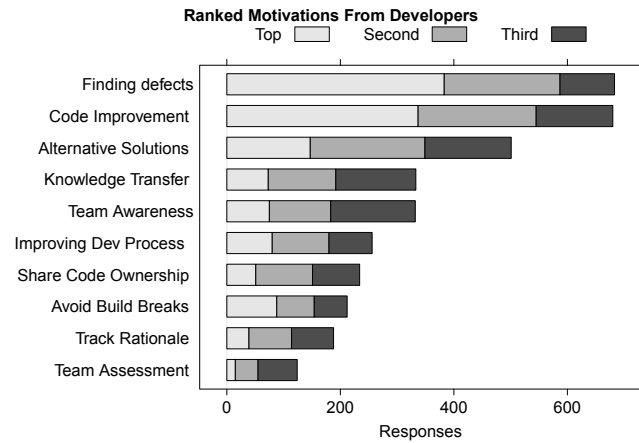


Fig. 3. Developers' motivations for code review.

"discipline of explaining your code to your peers [that] drives a higher standard of coding. I think the process is even more important than the result."

Most interviewed programmers mentioned that at least one of the reviewers involved in each code review takes care of checking whether the code follows the team conventions, for example in terms of code formatting and in terms of function and variable naming. Some programmers use a "code improvement" check as a first step when doing code review: *"the first basic pass on the code is to check whether it is standard across the team."*

The interviews also gave us a glimpse of the connection between the quality of code reviews and code improvement comments. Such comments seem easier to write and sometimes interviewees mentioned them as the way reviewers use to avoid spending time to conduct good code reviews. An observation by a senior developer, in the company for more than nine years, summarizes the opinions we received from many interviewees: *"I've seen quite a few code reviews where someone commented on formatting while missing the fact that there were security issues or data model issues."*

C. Alternative Solutions

Alternative solutions regard changes and comments on improving the submitted code by adopting an idea that leads to a better implementation. This is one of the few motivations in which developers and managers do not agree. While 147 (17%) developers put this as the first motivation, 202 (23%) as the second, and 152 (17%) as the third, only 4 (2%) managers even mentioned it (e.g., *"Generate better ideas, alternative approaches"* and *"Collective wisdom: Someone else on the project may have a better idea to solve a problem"*). The outcome of the interviews was similar to the position of managers: Interviewees vaguely mentioned this motivation, and mostly in terms of generic *"better ways to do things."*

D. Knowledge Transfer

All the interviewees but one motivated their code reviews also from a learning—or *knowledge transfer*—perspective. With the words of a senior developer: *"one of the things that should*

be happening with code reviews over time is a distribution of knowledge. If you do a code review and did not learn anything about the area and you still do not know anything about the area, then that was not as good code review as it could have been." Although we did not include questions related to *knowledge transfer* in our interview guideline, this topic kept emerging spontaneously from each meeting, thus underscoring its value for practitioners.

Sometimes programmers told us that they follow code reviews explicitly for learning purposes. For example, a tester explained: "[I read code reviews because] *from a code review you can learn about the different parts you have to touch to implement a certain feature.*"

According to interviewees, code review is a learning opportunity for both the author of the change and the reviewers: There is a bidirectional knowledge transfer about APIs usage, system design, best practices, team conventions, "*additional code tricks,*" *etc.* Moreover code reviews are recognized for educating new developers about code writing.

Managers included *knowledge transfer* as one of the reasons for code review, although never as the top motivation. They mostly wrote about code review as an education means by mentioning among the motivations: "*developer education,*" "*education for junior developers who are learning the codebase,*" and "*learning tool to teach more junior team members.*"

Programmers answering the survey declared *knowledge transfer* to be their first motivation for code review in 73 (8%) cases, their second in 119 (14%), and their third in 141 (16%).

E. Team Awareness and Transparency

During one of our observations, one developer was preparing a code review submission as an author: He wanted other developers to "*double check*" his changes before committing them to the repository. After preparing the code, he specified the developers he wanted to review his code; he required not only two specific people, but he also put a generic email distribution group as an "*optional*" reviewer. When we inquired about this choice, he explained us: "*I am adding [this alias], so that everybody [in the team] is notified about the change I want to do before I check it in.*" In the subsequent interviews, this concept of using an email list as optional reviewer, or including specific optional reviewers exclusively for awareness emerged again frequently, *e.g.*, "*Code reviews are good FYIs [for your information].*"

Managers often mentioned the concept of team awareness as a motivation for code review, frequently justifying it with the notion of "*transparency:*" Not only must the team be kept aware of the directions taken by the code, but also nobody should be allowed to "*secretly*" make changes that might break the code or alter functionalities.

The 873 programmers answering the survey ranked *team awareness and transparency* very close to *knowledge transfer*. In fact, the two concepts appeared logically related also in the interviews; for example one tester, while reviewing some code said: "*oh, this guy just implemented this feature, and now let me back and use it somewhere else.*" Showing that he both learned

about the new feature and he was now aware of the possibility to use it in his own code. 75 (9%) developers considered team awareness their first motivation for code review, 108 (12%) their second, and 149 (17%) their third.

Although *team awareness and transparency* emerged from our data as clearly promoted by the code review process, academic research seems to have given little attention to it.

F. Share Code Ownership

The concept of *shared code ownership* is closely related to *team awareness and transparency*, but it has a stronger connotation toward active collaboration and overlapping coding activities. Programmers and managers believe that code review is not only an occasion to notify other team members about incoming changes, but also a means to have more than one knowledgeable person about specific parts of the codebase. A manager put the following as her second motivation for code review: "*Broaden knowledge & understanding of how specific features/areas are designed and implemented (e.g., grooming "backup developers" for areas where knowledge is too concentrated on one or two expert developers).*"

Moreover, both developers and managers have the opinion that practicing code review also improves the personal perception of team members about shared code ownership. On this note, a senior developer, with more than 30 years in the software industry, explained: "*In the past people did not use to do code reviews and were very reluctant to put themselves in positions where they were having other people critiquing their code. The fact that code reviews are considered as a normal thing helps immensely with making people less protective about their code.*" Similarly a manager wrote us explaining that she deems code reviews important because they "*Dilute any "rigid sense of ownership" that might develop over chunks of code.*"

In the programmers' survey, 51 (6%) respondents marked *share code ownership* as their first motivation, 100 (11%) as their second, and 91 (10%) as their third.

G. Summary

In this section, we analyzed the motivations that developers and managers have for doing code review. We abstracted them into a list, which we finally included in the programmers' survey. Figure 3 reports the answers given to this question: The black bar is the number of developers that put that row as their top motivation, the gray bar is the number that put it as the second motivation, *etc.* We have ordered the factors by giving 3 points for a first motivation response, 2 points for a second motivation, *etc.* and then sorting by the sum.

We discussed the five most prominent motivations, which show that *finding defects* is the top motivation, although participants believe that code review brings other benefits. The first two motivations were already popular in research and their effectiveness have been evaluated in the context of code inspections; on the contrary, the other motivations are still unexplored, especially those regarding more social benefits on the team, such as shared code ownership.

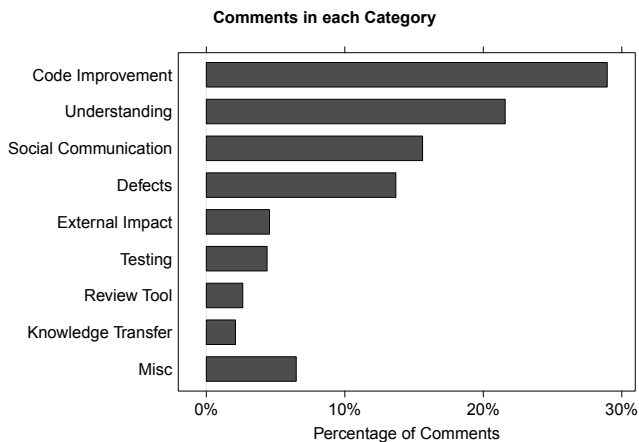


Fig. 4. Proportion of comments by card sort category.

Although motivations are well defined, we still have to verify whether they actually translate into real outcomes of a modern code review process.

V. THE OUTCOMES OF CODE REVIEWS

A. Motivations vs. Outcomes

Our second research question seeks to understand what the actual outcomes of code reviews are, and whether they match the motivations and expectations outlined in the previous section. To that end, we conducted indirect field research [24] by analyzing the content of 200 threads (corresponding to 570 comments) recorded by CodeFlow. Figure 4 shows the categories of comments found through the card sort.

Code Improvements: The most frequent category, with 165 (29%) comments, is *code improvements*. In detail, among *code improvements* comments we find 58 on using better code practices, 55 on removing not necessary or unused code, and 52 on improving code readability.

Defect Finding: Although *defect finding* is the top motivation and expected outcome of code review for many practitioners, the category *defect* is the only the fourth most frequent, out of nine items, with 78 (14%) comments. Among *defect* comments, 65 are on logical issues (e.g., a wrong expression in an if clause), 6 on high-level issues, 5 on security, and 3 on wrong exception handling.

Knowledge Transfer: Concerning the other expected outcomes of code reviews, we did not expect to find evidence about them, because of their more “social”—thus harder to quantifiy. Nevertheless, we found some (12) comments specifically about *knowledge transfer*, where the reviewers were directing the code change author to external resources (e.g., internal documentation or websites) for learning how to tackle some issues. This provides additional evidence on the importance of this aspect of reviews.

B. Finding Defects: When Expectations Do Not Meet Reality

Why do we see this significant gap in frequency between *code improvements* and *defects* comments? Possible reasons may be that our sample of 570 comments is too small to represent the population, that the submitted changes might

require less need fixing of “real” defects than of small code improvements, or that programmers could consider *code improvements* as actual defects. However, by triangulating these numbers with the interview discussions, the survey answers, and the other categories of comments, another reason seems to justify this situation. First, we start by noting that most of the comments on *defects* regard uncomplicated logical errors, e.g., corner cases, common configuration values, or operator precedence. Then, from interview data, we see that: (1) most interviewees explained how, with tool-based code reviews, most of the found defects regard “*logic issues—where the author might not have considered a particular or corner case*”; (2) some interviewees complained that the quality of code reviews is low, because reviewers only look for easy errors: “[Some reviewers] *focus on formatting mistakes because they are easy [...], but it doesn’t really help. [...] In some ways it’s kind of embarrassing if someone asks you to do a code review and all you can find are formatting mistakes when there are real mistakes to be found*”; and (3) other interviewees admitted that if the code is not among their codebase, they look at “*obvious bugs (such as, exception handling)*.” Finally, managers mentioned “*catching early obvious bugs*” or “*finding obvious inefficiencies or errors*” as reasons for doing code review. These points illustrate that the reason for the gap between the number of comments on *code improvements* and on *defects* is not to be found in problems in the sample or in classification misconceptions, but it is rather just additional corroborating evidence that the outcome of code review does not match the main expectation of both programmers and managers finding defects. Review comments about defects are few, comprising one-eighth of the total in our sample, and mostly address “micro” level and superficial concerns; while programmers and managers would expect more insightful remarks on conceptual and design level issues. Why does this happen? The high frequency of understanding comments hints at the answer to our question, addressed in the next section.

VI. WHAT ARE THE CHALLENGES OF CODE REVIEW?

Our third research question seeks to understand the main challenges faced by reviewers when performing modern code reviews, also with respect to the expected outcomes. We also seek to uncover the reasons behind the mismatch between expectations and actual outcomes on finding defects in reviews.

A. Code Review is Understanding

Even though we did not ask any specific question concerning understanding, the theme emerged clearly from our interviews. Many interviewees eventually acknowledged that understanding is their main challenge when doing code reviews. For example, a senior developer autonomously explained to us: “*the most difficult thing when doing a code review is understanding the reason of the change;*” a tester, in the same vein: “*the biggest information need in code review: what instigated the change;*” and another senior developer: “*in a successful code review submission the author is sure that his peers understand and approve the change.*” Although the textual description should

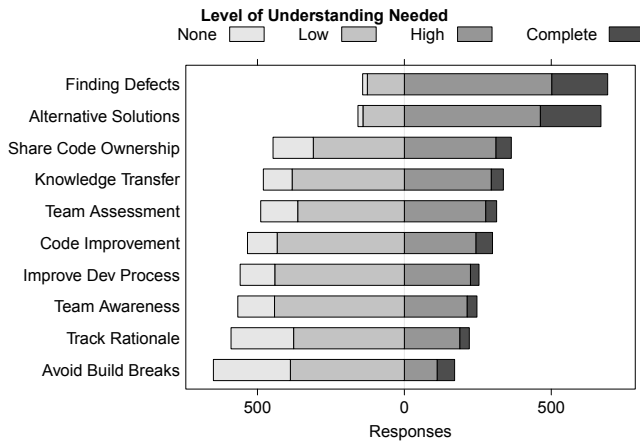


Fig. 5. Developers responses in surveys of the amount of code understanding for code review outcomes.

help reviewers understanding, some developers do not find it useful: “people can say they are doing one thing, while they are doing many more of them,” or “the description is not enough;” in general, developers seem to confirm that “not knowing files (or [dealing with] new ones) is a major reason for not understanding a change.”

From interviews, no other code review challenge emerged as clearly as understanding the submitted change. Even though scheduling and time issues also appeared challenging, we could always trace them back to the first challenge through the words of a tester: “understanding the code takes most of the reviewing time.” On the same note, in the code review comments we analyzed, the second most frequent category concerns understanding. This category includes clarification questions and doubts raised by the reviewers who want to grasp the rationale of the changes done on the code, and the corresponding clarification answers. This is also in line with the evidence delivered by Sutherland & Venolia on the relevance of rationale articulation in reviews [34].

Do understanding needs change with the expected outcome of code review? We included a question in the programmers’ survey to know how much understanding they needed to achieve each of the motivations listed in Figure 3. The outcome of the question is summarized in Figure 5. The respondents could answer with a four values Likert’s scale, by selecting the understanding of the change they felt was required to achieve the specific outcome. The most difficult task from the understanding perspective is *finding defects*, immediately followed by *alternative solutions*. Both clearly stand out from the other items. The gap in understanding needs between *finding defects* and *code improvement* seems to corroborate our hypothesis that the difference in the number of comments about these two items in review comments is mostly due to understanding issues. Thus, if managers and developers want code review to match their need for *finding defects*, context and change understanding must be improved.

B. Code Review is Understanding

By observing developers performing code reviews, we noticed that some started code reviews by thoroughly reading

the accompanying textual description, while others went directly to a specific changed file. In the first group, the time required for putting the first review comments and understanding the change rationale was noticeably longer, and some of the comments were asking to clarify the reasons for a change. To better comprehend this situation, we included in our interview guideline a question about how the interviewees start code reviews. Participants explained that when they own or are very familiar with the files being changed, they have a better context and it is easier for them to understand the change submitted: “when doing code review I start with things I am familiar with, so it is easier to see what is going on.” When they are file owners, they often do not need to read the description, but they “go directly to the files they own.” On the contrary, when they do not own files, or have to review new files, they need more information and try to get it from the description, which is deemed good when it states “what was changed and why.”

To better understand this aspect we included two questions in the programmers’ survey to know (1) whether it takes longer to review files they are not familiar with, and why; and (2) whether reviewers familiar with the changed files give different feedback, and how.

Most of the respondents (798, i.e., 91%) answered positively to the first question, motivating it with the fact that it takes time to familiarize with the code and “learn enough about the files being modified to understand their purpose, invariants, APIs, etc.,” because “big-picture impact analysis requires contextual understanding. When reviewing a small, unfamiliar change, it is often necessary to read through much more code than that being reviewed.” The comment of a developer anticipates the answer to the second question: “It takes a lot longer to understand unknown code, but even then understanding isn’t very deep. With code I am familiar with I have more to say. I know what to say faster. What I have to say is deeper. And I can be more insistent on it.” In fact, the answer to the second question is positive in 716 (82%) cases. The main difference with file owner comments is that they are substantially deeper, more detailed and insightful. A respondent explained: “Comments reflect their deeper understanding – more likely to find subtle defects, feedback is more conceptual (better ideas, approaches) instead of superficial (naming, mechanical style, etc.)” another tried to boldly summarize the concept: “Difference between algorithmic analysis and comments on coding style. The difference is big.” In fact, when the context is clear and understanding is very high, as in the case when the reviewer is the owner of changed files, code review authors receive comments that explore “deeper details,” are “more directed” and “more actionable and pertinent,” and find “more subtle issues.”

C. Dealing with Understanding Needs

From the interviews, we found that, in the current situation, reviewers try different paths to understand the context and the changes: They read the change description, try to run the changed code, send emails for understanding high level details about the review, and often (from 20% to 40% of the times) even go to talk in person to have a “higher communication

bandwidth” for asking clarifications to the author. All code review tools that we see in practice today deliver only basic support for the understanding needs of reviewers – providing features such as diffing capabilities, inline commenting, or syntax highlighting, which are limited when dealing with complex code understanding.

VII. RECOMMENDATIONS AND IMPLICATIONS

A. Recommendations for Practitioners

Although our work was revolving around a specific code review context (*i.e.*, code reviews with CodeFlow at Microsoft), we derive useful recommendations to developers, which can be generalized to other contexts:

Quality Assurance: There is a mismatch between the expectations and the actual outcomes of code reviews. From our study, review does not result in identifying defects as often as project members would like and even more rarely detects deep, subtle, or “macro” level issues. Relying on code review in this way for quality assurance may be fraught.

Understanding: When reviewers have *a priori* knowledge of the context and the code, they complete reviews more quickly and provide more valuable feedback to the author. Teams should aim to increase the breadth of understanding of developers (if the author of a change is the only expert, she has no potential reviewers) and change authors should include code owners and others with understanding as much as possible when using review to identify defects. Developers indicated that when the author provided context and direction to them in a review, they could respond better and faster.

Beyond Defects: Modern code reviews provide benefits beyond finding defects. Code review can be used to improve code style, find alternative solutions, increase learning, share code ownership, *etc.* This should guide code review policies.

Communication: Despite the growth of tools for supporting code reviews, developers still have need of richer communication than comments annotating the changed code when reviewing. Teams should provide mechanisms for in-person or, at least, synchronous communication.

B. Implications for Researchers

Our work uncovered aspects of code review—beyond our research questions—that deserve further study:

Automate Code Review Tasks: We observed that many code review comments were related to *code improvement* concerns and low-level “micro” defects. Identifying both of these are problems that research has begun to solve. Tools for enforcing team code conventions, checking for typos, and identifying dead code already exist. Even more advanced tasks such as checking boundary conditions or catching common mistakes have been shown to work in practice on real code. For example Google experimented with adding FindBugs to their review process, though little is reported about the results [5]. Automating these tasks frees reviewers to look for deeper, more subtle defects. Code review is fertile ground to have an impact with code analysis tools.

Program Comprehension in Practice: We identified context and change understanding as challenges that developers face when reviewing, with a direct relationship to the quality of review comments. Interestingly, modern IDEs ship with many tools to aid context and understanding, and there is an entire conference (ICPC) devoted to code comprehension, yet all current code review tools we know of show a highlighted diff of the changed files to a reviewer with no additional tool support. The most common motivation that we have seen for code comprehension research is a developer that is working on new code, but we argue that reviewers reviewing code they have not seen before may be more common than a developer working on new code. This is a ripe opportunity for code understanding researchers to have impact on real world scenarios.

Socio-technical Effects: Awareness and learning were cited as motivations for code review, but these outcomes are difficult to observe from traces in reviews. We did not investigate these further, but studies can be designed and carried out to determine if and how awareness and learning increase as a result of being involved in code review.

VIII. LIMITATIONS

As a qualitative study, gauging the validity of our findings is a difficult undertaking [17]. While we have endeavored to uncover and report the expectations, outcomes, and challenges of code review, limitations may exist. We describe them with the steps that we took to increase confidence and validity.

To achieve a comprehensive view of code review, we triangulated by collecting and comparing results from multiple sources. For example, we found strong agreement among the results of expectations collected from interviews, surveys of manager, and surveys of developers. By starting with exploratory interviews of a smaller set of subjects (17) followed by open coding to extract themes, we identified core questions that we addressed to a larger audience via survey.

One potential criticism is that empirical research within one company or one project provides little value for the academic community, and does not contribute to scientific development. Historical evidence shows otherwise. Flyvbjerg provides several examples of individual cases that contributed to discovery in physics, economics, and social science [14]. Beveridge observed for social sciences: “*More discoveries have arisen from intense observation than from statistics applied to large groups*” (as quoted in Kuper and Kuper [21], page 95). This should not be interpreted as a criticism of research that focuses on large samples. For the development of an empirical body of knowledge as championed by Basili [8], both types of research are essential. To understand code review across many contexts, we observed, interviewed, surveyed, and examined code reviews from developers across a diverse group of software teams that work with codebases in various domains, of varying sizes, and with varying processes.

Concerning the representativeness of our results in other contexts, other companies and OSS use tools similar to CodeFlow [40], [36], [19]. However, team dynamics may differ. The need for code understanding may already be met

in contexts where projects are smaller or there is shared code ownership and a broad system understanding across the team. We found that higher levels of understanding lead to more informative comments, which identify defects or aid the author in other ways so review in these contexts may uncover more defects. In OSS contexts, project-specific expertise often must be demonstrated prior to being accepted as a “core committer” [10], so learning may not be as important or frequent an outcome for review.

In this work, we have used discussions within CodeFlow to identify and quantify outcomes of code review. However, some motivations that managers and developers described are not easily observable because they leave little trace. For example, determining how often code review improves team awareness or transfers knowledge is difficult to assess from the discussions in reviews. For these outcomes, we have responses indicating that they occur, but not “hard evidence.”

Based on review comments, survey responses, and interviews, we know that in-person discussions occurred frequently. While we cannot compare frequency of these events to other outcomes as we can with events recorded in CodeFlow, we know that they most often occurred to address understanding needs.

IX. CONCLUSION

We investigated modern, tool-based code review, uncovering both a wide range of motivations for review and that the outcomes do not always match those motivations. We identified understanding as a key component and provided recommendations to both practitioners and researchers. It is our hope that the insights we have discovered lead to more effective review in practice and improved tools, based on research, to aid developers perform code reviews.

REFERENCES

- [1] A. Ackerman, L. Buchwald, and F. Lewski. Software inspections: An effective verification process. *Software, IEEE*, 6(3):31–36, 1989.
- [2] A. Ackerman, P. Fowler, and R. Ebenau. Software inspections and the industrial production of software. In *Proc. of a symposium on Software validation: inspection-testing-verification-alternatives*, pages 13–40. Elsevier North-Holland, Inc., 1984.
- [3] J. Adair. The Hawthorne effect: A reconsideration of the methodological artifact. *Journal of applied psychology*, 69(2):334, 1984.
- [4] S. Adolph, W. Hall, and P. Kruchten. Using grounded theory to study the experience of software development. *Empirical Software Engineering*, 16(4):487–513, 2011.
- [5] N. Ayewah, W. Pugh, J. Morgenthaler, J. Penix, and Y. Zhou. Using findbugs on production software. In *Companion to the 22nd ACM SIGPLAN conference on Object-oriented programming systems and applications companion*, pages 805–806. ACM, 2007.
- [6] A. Bacchelli and C. Bird. Appendix to expectations, outcomes, and challenges of modern code review. <http://research.microsoft.com/apps/pubs/?id=171426>, Aug. 2012. Microsoft Research, Technical Report MSR-TR-2012-83 2012.
- [7] I. Barker. What is information architecture? <http://www.steptwo.com.au/>, May 2005.
- [8] V. Basili, F. Shull, and F. Lanubile. Building knowledge through families of experiments. *IEEE Trans. on Software Eng.*, 25(4):456–473, 1999.
- [9] B. Berg and H. Lune. *Qualitative research methods for the social sciences*. Pearson Boston, 2004.
- [10] C. Bird, A. Gourley, P. Devanbu, A. Swaminathan, and G. Hsu. Open borders? immigration in open source projects. In *The Fourth International Workshop on Mining Software Repositories*, 2007.
- [11] L. Brothers, V. Sengugamoorthy, and M. Muller. Icicle: groupware for code inspection. In *Proceedings of the 1990 ACM conference on Computer-supported cooperative work*, pages 169–181. ACM, 1990.
- [12] J. Creswell. *Research design: Qualitative, quantitative, and mixed methods approaches*. Sage Publications, 3rd edition, 2009.
- [13] M. Fagan. Design and code inspections to reduce errors in program development. *IBM Systems Journal*, 15(3):182–211, 1976.
- [14] B. Flyvbjerg. Five misunderstandings about case-study research. *Qualitative inquiry*, 12(2):219–245, 2006.
- [15] J. Gintell, J. Arnold, M. Houde, J. Kruszelnicki, R. McKenney, and G. Memmi. Scrutiny: A collaborative inspection and review system. *Software Engineering/ESEC’93*, pages 344–360, 1993.
- [16] B. Glaser. *Doing Grounded Theory: Issues and Discussions*. Sociology Press, 1998.
- [17] N. Golafshani. Understanding reliability and validity in qualitative research. *The qualitative report*, 8(4):597–607, 2003.
- [18] J. Johnson. Collaboration, peer review and open source software. *Information Economics and Policy*, 18(4):477–497, 2006.
- [19] N. Kennedy. How google does web-based code reviews with mondrion. <http://www.test.org/doel/>, Dec. 2006.
- [20] B. Kitchenham and S. Pfleeger. Personal opinion surveys. *Guide to Advanced Empirical Software Engineering*, pages 63–92, 2008.
- [21] A. Kuper. *The social science encyclopedia*. Routledge, 1995.
- [22] O. Laitenberger. A survey of software inspection technologies. *Handbook on Software Engineering and Knowledge Engineering*, 2:517–555, 2002.
- [23] T. LaToza, G. Venolia, and R. DeLine. Maintaining mental models: a study of developer work habits. In *Proceedings of the 28th international conference on Software engineering*, pages 492–501. ACM, 2006.
- [24] T. Lethbridge, S. Sim, and J. Singer. Studying software engineers: Data collection techniques for software field studies. *Empirical Software Engineering*, 10(3):311–341, 2005.
- [25] V. Mashayekhi, C. Feulner, and J. Riedl. Cais: collaborative asynchronous inspection of software. In *ACM SIGSOFT Software Engineering Notes*, volume 19, pages 21–34. ACM, 1994.
- [26] A. Porter, H. Siy, and L. Votta. A review of software inspections. *Advances in Computers*, 42:39–76, 1996.
- [27] T. Punter, M. Ciolkowski, B. Freimut, and I. John. Conducting on-line surveys in software engineering. In *International Symposium on Empirical Software Engineering*. IEEE, 2003.
- [28] P. Rigby, B. Cleary, F. Painchaud, M. Storey, and D. German. Open source peer review—lessons and recommendations for closed source. *IEEE Software*, 2012.
- [29] P. Rigby, D. German, and M. Storey. Open source software peer review practices: a case study of the apache server. In *Proceedings of the 30th international conference on Software engineering*. ACM, 2008.
- [30] P. C. Rigby and M.-A. Storey. Understanding broadcast based peer review on open source software projects. In *Proceedings of ICSE 2011 (33rd International Conference on Software Engineering)*, pages 541–550. ACM, 2011.
- [31] J. E. Shade and S. J. Janis. *Improving Performance Through Statistical Thinking*. McGraw-Hill, 2000.
- [32] F. Shull and C. Seaman. Inspecting the history of inspections: An example of evidence-based technology diffusion. *Software, IEEE*, 25(1):88–90, 2008.
- [33] M. Stein, J. Riedl, S. J. Harner, and V. Mashayekhi. A case study of distributed, asynchronous software inspection. In *Proceedings of the international conference on Software engineering*. ACM, 1997.
- [34] A. Sutherland and G. Venolia. Can peer code reviews be exploited for later information needs? In *International Conference on Software Engineering, New Ideas and Emerging Results Track*, 2009.
- [35] B. Taylor and T. Lindlof. *Qualitative communication research methods*. Sage Publications, Incorporated, 2010.
- [36] A. Tsotsis. Meet phabricator, the witty code review tool built inside facebook. <http://techcrunch.com/2011/08/07/oh-what-noble-scribe-hath-penned-these-words/>, Aug. 2006.
- [37] P. Tyagi. The effects of appeals, anonymity, and feedback on mail survey response patterns from salespeople. *Journal of the Academy of Marketing Science*, 17(3):235–241, 1989.
- [38] L. Votta Jr. Does every inspection need a meeting? *ACM SIGSOFT Software Engineering Notes*, 18(5):107–114, 1993.
- [39] R. Weiss. *Learning from strangers: The art and method of qualitative interview studies*. Simon and Schuster, 1995.
- [40] Wikipedia. Gerrit (software). [http://en.wikipedia.org/wiki/Gerrit_\(software\)](http://en.wikipedia.org/wiki/Gerrit_(software)), June 2012.